

Автоматизация выявления перспективных геохимических классов и их положений на карте методом корреляционного анализа

Баранов Владислав

СпбПУ

18 декабря 2024 г.

Постановка задачи

- **Описание задачи:** На данный момент процесс анализа геохимических проб занимает достаточно большое количество времени, однако ход работы остается неизменным.
- **Цель работы:** Автоматизировать процесс корреляционного анализа с помощью скрипта на языке python. Сформировать набор данных, которые пригодны для последующего анализа специалистом.

Особенности задачи

- **Входные данные:** представляют собой таблицу, состоящую из точек пробоотбора, для каждой из которых определено положение на карте и содержание химических элементов.
Также, в связи со своим естественным происхождением, данные имеют логнормальное распределение.
- **Выходные данные:** таблица Excel с двумя страницами – результатами кластеризации и генеральными характеристиками исходного набора данных. Карта распределения проб на карте. Эти данные позволяют специалисту проанализировать и понять, выделяются ли геохимические ассоциации и есть ли среди них потенциально-золоторудные.

Входные данные

№ проб	x msk	y msk	Ag	As	B	...	Sn	V	W	Zn	Au
1	120900	204480	1.70	40.0	36	...	8.0	210.0	4.1	140.0	0.004
2	119900	204481	0.65	2.5	29	...	11.0	55.0	1.8	2.5	0.001
3	120101	204481	1.20	1700.0	180	...	600.0	160.0	21.0	220.0	0.005
4	119898	204501	1.80	82.0	110	...	74.0	140.0	2.6	180.0	0.100
5	120900	204520	1.70	160.0	51	...	9.8	375.0	4.7	250.0	0.001
6	119996	204521	0.24	28.0	60	...	310.0	53.0	3.4	60.0	0.001
7	119998	204539	0.70	130.0	180	...	600.0	120.0	3.2	140.0	0.001
8	120900	204540	2.80	140.0	50	...	11.0	375.0	4.0	230.0	0.002
9	119901	204561	0.38	17.0	37	...	58.0	81.0	2.0	2.5	0.001
10	120000	204561	4.70	140.0	180	...	150.0	210.0	4.4	1900.0	0.001
11	119802	204561	3.70	150.0	180	...	350.0	89.0	2.1	400.0	0.005
12	120306	204562	1.90	89.0	110	...	14.0	200.0	2.2	380.0	0.008
13	120401	204600	1.50	33.0	39	...	19.0	200.0	4.0	370.0	0.001
14	120201	204601	1.90	110.0	59	...	100.0	160.0	1.6	360.0	0.001
15	120101	204602	0.97	760.0	80	...	87.0	375.0	2.1	260.0	0.039
16	119806	204603	0.24	2.5	24	...	8.0	42.0	1.1	2.5	0.001
17	119600	204619	0.38	15.0	180	...	53.0	88.0	2.2	300.0	0.001
18	120402	204619	1.90	64.0	64	...	17.0	375.0	3.9	760.0	0.001
19	120302	204621	0.62	5.6	17	...	6.5	61.0	1.9	67.0	0.003
20	120103	204639	0.83	33.0	39	...	63.0	200.0	2.3	97.0	0.085

Использованные библиотеки

- **numpy**: Используется для математических вычислений и обработки массивов.Используется для математических вычислений и обработки массивов.
- **pandas**: Применяется для работы с табличными данными, их обработки и анализа.
- **sklearn**: Используется для задач машинного обучения, включая нормализацию данных и кластеризацию.
- **plotly**: Предназначена для создания интерактивных графиков и визуализации данных.

Структура программы

```
if __name__=='__main__':  
    # Параметры  
    path = 'bd_float.xlsx'  
    required_level = 'second'  
    c = 11 # Количество кластеров  
    i = 300 # Количество итераций  
  
    # Загрузка данных  
    raw_data = pd.read_excel(path, index_col=0)  
    log_data = raw_data.apply(np.log)  
  
    # Расчет статистик  
    general_stats = get_stats(log_data)  
  
    # Кластеризация  
    clusters = get_kmeans_clusters(raw_data, general_stats, c, i)  
    raw_data['cluster'] = clusters
```

Расчет статистик

```
def get_stats(log_data):  
    stats = log_data.describe().T  
    stats['mode'] = log_data.mode().iloc[0]  
    stats['skew'] = log_data.skew()  
  
    stats = stats.apply(np.exp)  
  
    stats['cf.var(%)'] = stats['std'] / stats['mean'] * 100  
    stats['first'] = stats['mean'] * stats['std']  
    stats['second'] = stats['first'] * stats['std']  
    stats['third'] = stats['second'] * stats['std']  
  
    return stats
```

Кластеризация k-средних - это один из наиболее распространенных методов машинного обучения, который используется для разбиения множества данных на группы (кластеры) на основе сходства объектов между собой. Он используется для автоматической группировки данных на основе их характеристик и свойств, которые описывают объекты.

Обоснование выбора метода k-средних

- **Преимущества метода k-средних:**

- Простота и эффективность вычислений, особенно для больших наборов данных.
- Хорошо подходит для задач, где требуется выделить компактные, сферические кластеры.
- Возможность легко интерпретировать результаты (центроиды кластеров отражают средние значения характеристик).

- **Почему не другие методы?**

- *Иерархическая кластеризация*: Требует больших вычислительных ресурсов при увеличении объема данных.
- *DBSCAN*: Менее подходит для данных с логнормальным распределением и может требовать сложной настройки параметров.
- *Gaussian Mixture Models (GMM)*: Сложнее в интерпретации и требует больше времени на обучение.

Кластеризация методом k-средних

```
def get_kmeans_clusters(raw_data, general_stats, c, i):  
    data = raw_data.drop(['Hg', 'Li', 'Cr', 'x msk', 'y msk'],  
                          axis=1).astype(float)  
  
    # Нормализация данных  
    scaler = StandardScaler()  
    scaler.fit(data)  
    scaled_features = scaler.transform(data)  
    scaled_data = pd.DataFrame(scaled_features, columns=data.columns)  
  
    # Кластеризация данных  
    k_means = KMeans(random_state=10, n_init=i, n_clusters=c)  
    k_means.fit(scaled_data)  
    labels = k_means.labels_  
    return labels
```

Использование правила трех сигм

- **Что такое правило трех сигм?**

- В статистике правило трех сигм предполагает, что практически все значения (около 99.7%) нормального распределения находятся в пределах трех стандартных отклонений ($\pm 3\sigma$) от среднего.

- **Как это используется в проекте?**

- Для выявления выбросов в геохимических данных.
- Данные, выходящие за пределы, рассматриваются как аномальные.
- Аномальные значения заменяются на пороговые.

- **Почему это важно?**

- Исключение выбросов предотвращает их влияние на процесс кластеризации, который чувствителен к аномальным данным.
- Улучшается качество кластеров, что критично для геохимических исследований.

- **Пример:**

- Если содержание элемента в пробах превышает значение $\mu + 3\sigma$, его можно заменить на пороговое значение $\mu + 3\sigma$.

Что такое нормализация данных?

- **Определение:**

- Нормализация данных — это процесс преобразования данных в стандартный масштаб, где среднее значение каждой переменной равно 0, а стандартное отклонение равно 1.

- **Зачем это нужно?**

- Для устранения влияния масштаба признаков на результаты алгоритмов машинного обучения.
- Ускоряет сходимость алгоритмов, таких как метод k-средних.

- **Как это работает в sklearn?**

- ① `scaler = StandardScaler()` — создаётся объект нормализатора.
- ② `scaler.fit(data)` — вычисляются параметры (среднее и стандартное отклонение) на основе входных данных.
- ③ `scaled_features = scaler.transform(data)` — данные преобразуются с использованием ранее вычисленных параметров.

- **Результат:**

- Все признаки имеют одинаковый масштаб, что улучшает производительность алгоритмов кластеризации и классификации.

Тепловая карта

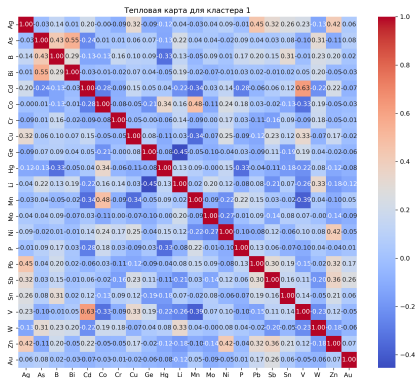


Рис.: Тепловая карта для 1 кластера

Почему выбрано 300 итераций?

- Метод k-средних является итеративным алгоритмом, который стремится минимизировать сумму квадратов расстояний от точек до центроидов кластеров.
- **Выбор 300 итераций:**
 - Это значение достаточно велико, чтобы обеспечить сходимость алгоритма в большинстве случаев.
 - Увеличение числа итераций после 300 обычно приводит к незначительному улучшению качества кластеризации.
- **Если итераций меньше:**
 - Алгоритм может не успеть сойтись к оптимальным значениям центроидов.
 - Это может привести к менее точному разбиению данных.
- **Если итераций больше:**
 - Увеличивается время выполнения алгоритма без значительного улучшения результата.
 - Может появиться риск перерасхода вычислительных ресурсов.

Сводная таблица

```
def get_pivot_table(raw_data):  
    pivot = raw_data.drop(['x msk', 'y msk'],  
                           axis=1).pivot_table(index='cluster',  
                                                aggfunc='mean')  
    pivot['count'] = raw_data['cluster'].value_counts()  
    return pivot
```

Карта распределения проб на карте

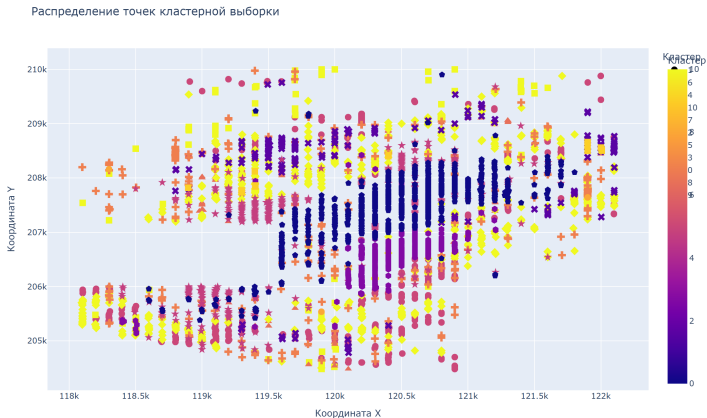


Рис.: Карта распределения проб на карте

Rand Index для оценки качества кластеризации

Для проверки качества кластеризации относительно эталонной кластеризации, выполненной в программе Statistica 10 использован индекс Rand, который измеряет сходство двух назначений кластеров, игнорируя перестановки. Учитывая что идеальная маркировка оценивается в 1,0, индекс 0,83 подтверждает схожесть кластеров.



Рис.: Rand Index

Обоснование использования Rand Index

- **Особенности задачи:**

- Задача требует оценки сходства кластеризации геохимических данных с эталонными метками.
- Необходимо использовать метрику, которая учитывает совпадения кластеров по парам объектов.

- **Преимущества Rand Index (RI):**

- Простота расчета и интерпретации.
- Подходит для сравнения любых двух разбиений объектов.
- Учитывает как положительные (одинаковые кластеры), так и отрицательные совпадения (разные кластеры).

- **Почему не Adjusted Rand Index (ARI)?**

- ARI корректирует случайные совпадения, но это избыточно для данной задачи, где метки эталона известны и не случайны.
- RI достаточен для анализа геохимических данных, так как точные значения случайных совпадений не играют критической роли.

- **Результаты работы:**

- Разработан скрипт на языке Python для автоматизации корреляционного анализа геохимических данных.
- Использованы современные методы кластеризации и нормализации данных, включая метод k-средних.
- Реализована модульная структура программы для удобства дальнейшего развития и адаптации.

- **Преимущества автоматизации:**

- Значительное сокращение времени анализа данных.
- Повышение точности и воспроизводимости результатов.

- **Направления для дальнейших исследований:**

- Апробация метода на других наборах данных.
- Оптимизация алгоритмов для работы с большими объемами данных.
- Интеграция дополнительных методов визуализации для более детального анализа.

Спасибо за внимание!

Вопросы?